A Unified Formalism for the Automated Verification of Arbitrary Program Logic

Paul Schmeing

March 2026

1 Abstract

The modern low level software development is plagued by an inherent gap between implementation and formal verification. Manual audits are tedious and economically expensive, while current security mechanisms focus on only one problem, like "Memory Safety". In this paper, we present **Otter**, a unified formalism for the automated verification of arbitrary program logic.

The core of the system is the **Otter Abstract Machine (OAM)**, which represents the program state in a directed graph consisting of domains. The abstraction of the physical hardware limits to provable, finite resource limits allows the creation of **Otter Proof Archives (OPA)**. This artifact ensures a cryptographic verified prove for the absence of runtime errors, undefined behavior, and logical invariant validations. With this, Otter transforms the audit-process from a reactive, human examination to a proactive mathematical certainty.

2 The Otter Abstract Machine (OAM)

The OAM is defined as a triple $\mathcal{M} = (G, \Sigma, \delta)$, where:

- G is the directed Property Graph representing all active domains and their relations.
- Σ is the Alphabet of Operations (Transitions).
- δ is the Transition Function $\delta : G \times \Sigma \to G \cup \{\bot\}$, where $\bot$ represents an unprovable (invalid) state.

For reasons of practicality the OAM abstracts away specific hardware constraints, like memory and time, to focus on computational decidability and logical integrity. This means every prove has to be executable in a finite amount of time and use a finite amount of memory, not considering if the resulting hardware requirements are practical.

3 Domain Definition & Taxonomy

In the OAM, a domain $\mathcal{D}$ is considered the smallest unit of logical verification. We differentiate, between four fundamental classes of domains.

3.1 Primitive Domain ($\mathcal{D}_{atomic}$)

This domain is a standard domain as defined in the Set Theory. It describes a set of scalar values $\mathbb{S}$.

$$\mathcal{D}_{atomic} = \{v \mid v \in \mathbb{S} \land P(v)\} \tag{1}$$

where P is a predicate, for example an Interval $[0, 100]$ or disjunct values $\{2, 3, 5, 7\}$.

3.2 Ordered Domain ($\mathcal{D}_{ordered}$)

This type of domain is primarily used to describe complex data structures. The domain is defined as a tuple of n domains.

$$\mathcal{D}_{ordered} = (\mathcal{D}_1, \mathcal{D}_2, ..., \mathcal{D}_n) \tag{2}$$

Accessing a member m_i of an ordered domain $\mathcal{D}_{ordered}$ yields the sub-domain $\mathcal{D}_i$ while maintaining the constraints of the parent domain.

3.3 Proxy Domain ($\mathcal{D}_{proxy}$)

This type of domain is used to model references to another domain $\mathcal{D}_{target}$. A $\mathcal{D}_{proxy}$ is a singleton set containing the unique identifier ι of a target domain $\mathcal{D}_{target}$ within the OAM state graph.

$$\mathcal{D}_{proxy} = G_\iota \tag{3}$$

3.4 Mapped Domain ($\mathcal{D}_{mapped}$)

This is arguably the most complex type of domain used in the OAM, it can be considered similar to a dependent type. It is used to define the domain depending on a different domain. It is a set of ordered domains containing D_{key} which describes the condition for this mapping and D_{value} which describes the domain.

$$\mathcal{D}_{mapped} = \{(\mathcal{D}_{key,i}, \mathcal{D}_{value,i}) \mid i \in I\} \tag{4}$$

A transition into a mapped domain is only valid if the current state s satisfies at least one $\mathcal{D}_{key,i}$, when more than one are satisfied only the intersection of the domains is accessible until further refinement. When more $\mathcal{D}_{key,i}$ are satisfied the resulting domain is defined as

$$I(x) = \{i \in I \mid x \in \mathcal{D}_{key,i}\}$$
$$\mathcal{D}_{mapped} = \bigcap_{i \in I(x)} \mathcal{D}_{value,i} \tag{5}$$

3.5 Invalid Domain

A domain is considered invalid when the values it represents are no longer accessible or were destroyed. An invalid domain cannot be interacted with, if an access is tried this is considered to make this section unprovable and thus disproves the current check.

Regarding $\mathcal{D}_{proxy}$ this means that when $\mathcal{D}_{target}$ is invalid the proxy domain is automatically considered invalid, and neither can be used.

3.6 Structs

The OAM understands structs and considers them as to be an ordered domain ($\mathcal{D}_{ordered}$), the domain of a struct is a cartesian product of all domains it includes.

4 The Proof Graph

4.1 State transitions

4.1.1 Allowed Transitions

4.2 Mapping logical memory to physical memory

5 Formal Semantics

A program P is considered proven if for all inputs $E \in \text{Domain}(E)$ the path through the OAM is finite and no invariant is violated.

The time-complexity $T(n)$ is not considered in the OAM. A program with $T(n) = O(2^n)$ is considered as secure as an identical program with $T(n) = O(1)$, as long as both terminate and do not violate any constraints.

6 Otter Prove Archive (OPA)

6.1 Structure & Contents

6.2 Serialization

6.3 Signing